

SolarMax

Setting the Clock — One-Time Wi-Fi Time Setup

Do this once, right after flashing. The clock keeps itself afterward.

Contents

1	Where you are, and what this fixes	1
2	What you need	1
3	Step 1 — Put your Wi-Fi details in the config	1
4	Using a phone hotspot (optional, easiest)	2
5	Step 2 — Upload the real firmware	2
6	Step 3 — Watch it get the time (Serial Monitor)	3
7	Step 4 — Done. The clock is set for good	3
8	Troubleshooting	3

1. Where you are, and what this fixes

You have already flashed the ESP32 — nice work. When you run the *real* firmware and open the Serial Monitor, you're seeing this on repeat:

```
[ERROR] No valid time source - fix WiFi/RTC and reset.
```

This is not a bug. SolarMax figures out where the sun is from the current **date and time**, so if it doesn't trust its clock it refuses to move the panel (it fails safe). The board just doesn't have the time *yet*.

This guide does the one-time fix: connect the board to Wi-Fi **once**, it grabs the time from the internet and writes it into the DS3231 real-time clock (RTC). After that the RTC keeps time on its own little battery — **no Wi-Fi needed again**.

Note

The RTC is already wired on your board. You are not changing any wiring here — only giving the board internet access one time so it can learn the correct time.

2. What you need

- The flashed ESP32, plugged in by USB (same as when you flashed).
- A **2.4 GHz** Wi-Fi network *or* a phone hotspot (see below).

- A coin-cell battery (CR2032) seated in the DS3231 module.

Important

The ESP32 can **only** join **2.4 GHz** Wi-Fi — it cannot see 5 GHz networks. Most home routers broadcast both; if joining fails, that's the #1 reason. A phone hotspot set to 2.4 GHz is the easiest guaranteed option.

3. Step 1 — Put your Wi-Fi details in the config

1. In VS Code, open the file `include/config.h`.
2. Go to **lines 9–10** (the `WIFI_SSID` and `WIFI_PASS` lines):

```
#define WIFI_SSID    "YOUR_SSID"  
#define WIFI_PASS    "YOUR_PASSWORD"
```

3. Replace them with your network name and password (keep the quotes):

```
#define WIFI_SSID    "MyNetwork24"  
#define WIFI_PASS    "mypassword123"
```

4. Save the file (Ctrl+S).

Important

The network name and password are **case-sensitive** — type them exactly. Please **don't commit your real password** to GitHub; change these two lines back to placeholders before pushing, or just don't push `config.h`.

4. Using a phone hotspot (optional, easiest)

If you don't have easy 2.4 GHz Wi-Fi, a phone hotspot works great — just force it to 2.4 GHz, then use its name/password in Step 1.

Phone hotspot

iPhone: Settings → Personal Hotspot → turn on **Maximize Compatibility** (this forces 2.4 GHz). Turn the hotspot **on** and keep that screen open while the board connects.

Android: Settings → Hotspot & tethering → Wi-Fi hotspot → set **AP Band** to **2.4 GHz**. Turn it on.

Tip

Make sure the phone providing the hotspot has **mobile data** on — the board needs to reach the internet to read the time, not just join the hotspot.

5. Step 2 — Upload the real firmware

You'll re-upload, this time the **real** firmware environment, `esp32dev`. Two ways — pick one:

A. From the PlatformIO sidebar (click-based):

1. Click the **alien-head** icon in the left bar to open PlatformIO.
2. Expand **Project Tasks** → **esp32dev** → **General**.
3. Click **Upload**. Wait for [SUCCESS] in the terminal.

B. From the PlatformIO terminal (one command):

1. Open the PlatformIO CLI terminal (the terminal where the `pio` command works).
2. Run:

```
pio run -e esp32dev -t upload
```

Note

This is the *real* firmware (`esp32dev`), not the on-board simulation (`simulate_hw`) you may have run before. Make sure you pick `esp32dev`.

6. Step 3 — Watch it get the time (Serial Monitor)

Open the Serial Monitor (the **plug** icon in the PlatformIO bottom toolbar, or run `pio device monitor`). Right after the board resets you should see it find the RTC, join Wi-Fi, and set the clock:

```
[RTC] DS3231 found
[NTP] Connecting to WiFi... - connected
[NTP] Syncing time... - OK
[RTC] Updated from NTP
```

That [RTC] Updated from NTP line is the whole goal: the correct time is now stored in the RTC. The `No valid time source` error is gone, and the tracker starts running.

Tip

It tries Wi-Fi for about 10 seconds. If it can't connect it says **FAILED (timeout)** — that's the moment to double-check 2.4GHz, the name/password, and that the hotspot is on (see Troubleshooting).

7. Step 4 — Done. The clock is set for good

Once you've seen [RTC] Updated from NTP, you're finished:

- You can **turn off the hotspot / Wi-Fi**. On every future power-up the board reads the time from the RTC and prints:

```
[RTC] Time valid from battery backup
```

...then tracks normally, **no Wi-Fi required**.

- **Re-flashing does not erase the clock.** The RTC is a separate chip with its own battery, so uploading new code later keeps the correct time.

Tip

You may leave your real Wi-Fi details in `config.h` if you like — the firmware does a small daily re-sync when Wi-Fi is reachable to stay perfectly accurate, and simply falls back to the RTC when it isn't. It's optional; the DS3231 is accurate to about a minute per year on its own.

8. Troubleshooting

- **Still “No valid time source” / Wi-Fi FAILED (timeout):** the board couldn't join the network. Check that it's **2.4 GHz** (not 5 GHz), the SSID/password are exact (case-sensitive), and the hotspot is on with mobile data.
- **Wi-Fi connects but Syncing time... FAILED:** it joined but couldn't reach the internet clock. Use a hotspot that has working mobile data, or a network that isn't blocking outbound time (NTP) traffic.
- **It worked, but after unplugging it's back to “not valid”:** the RTC's coin battery is dead or missing — it can't remember the time without it. Fit a fresh **CR2032** and set it once more.
- **[RTC] DS3231 not detected:** the clock module isn't answering on I²C. Check its wiring (`SDA = GPIO21`, `SCL = GPIO22`, `3V3`, `GND`); the `bringup` build's I²C scan should list a device at `0x68`.